

Vilnius University Šiauliai Academy



Object Oriented Programming
Practical Work 3:
Inheritance, constants

Lecturer: Dr. Donatas Dervinis
Student: Nikola Odjakov

Šiauliai, 2026

Test Report

Task

1. Create a child class (inheritance) based on the class of the previous task ($3x+1$) by adding new method which can calculate statistics (histogram) of function $3x+1$ iteration values from interval $[n;m]$
2. Set correct variable modifiers (public, private...);
3. Use 2-3 constants values.

Clarification

The logic and input handling of the userCollatzClass and subclass.php remain the same as the previous versions. Therefore, the results of the test cases for the intervals, Collatz calculation, will remain the same. In this iteration, inheritance is added in by creating a new method to calculate statistics (histogram) of the collatz conjecture's results of the user given iteration values for the intervals $n:m$. Besides that, variable modifiers are integrated (public, private) as well as 4 constants. The tests conducted therefore will be in regards to these newly added features.

Utilization of Constant values

```

/ references | 1 implementation
class userCollatz {
protected $startingUserNum,$startingUserIntervalNum,$endingUserIntervalNum,$cumulativeIntervalValuesStatistics;
1 reference
const REACHED_COLLATZ = 1;
2 references
const EVEN_CASE_DIVIDE =2;
1 reference
const ODD_CASE_MULTIPLY=3;
1 reference
const ODD_CASE_INCREASE=1;
...

0 references | 0 overrides
public function __construct($startingUserNum) {
    $this->startingUserNum= $startingUserNum;
}

2 references | 0 overrides
public function collatzCalculate() {
    $startingUserNum = $this->startingUserNum;
    $listCollatz = [$startingUserNum];

    while ($startingUserNum != self::REACHED_COLLATZ){
        if ($startingUserNum % self::EVEN_CASE_DIVIDE == 0) {
            $startingUserNum = $startingUserNum / self::EVEN_CASE_DIVIDE;
        } else {
            $startingUserNum = self::ODD_CASE_MULTIPLY * $startingUserNum + self::ODD_CASE_INCREASE;
        }

        $listCollatz[] = $startingUserNum;
    }
    return $listCollatz;
}

```

As per task 3, constants with clear naming schemes were used to improve the readability of the program. As a preventative check of whether the program still runs, the program is initiated, and collatzCalculate is ran:

Test 1:

```

C:\Users\Acer\Desktop\3xPlus1_OOP_Inheritance_constants_Assignment3>php C:\Users\Acer\Desktop\3xPlus1_OOP_Inheritance_constants_Assignment3\3xplusloop.php
--Welcome, here is a practical demonstration of the Collatz sequence!--
Enter a number to receive Collatz (number must be >= 1): 10
Sequence step 1: 10
Sequence step 2: 5
Sequence step 3: 16
Sequence step 4: 8
Sequence step 5: 4
Sequence step 6: 2
Sequence step 7: 1
Enter the first interval number for Collatz:

```

Result: As shown in the figure above, the program successfully initiated, and calculated the Collatz sequences correctly, as well as proceeding to the next step.

Utilization of variable modifiers

Variable modifiers are utilized in two instances; in the histogram class, as well as the Collatz interval class.

```
class Histogram extends userCollatz {  
  
    1 reference | 0 overrides  
    public function createHistogramDisplay($intervalCollatz) {  
        $histogram = [];  
        foreach ($intervalCollatz as $histoindex => $histovalues) {  
            $iterationCount = count($histovalues) - 1;  
            if (array_key_exists($iterationCount, $histogram)) {  
                $histogram[$iterationCount]++;  
            }  
            else {  
                $histogram[$iterationCount] = 1;  
            }  
        }  
        arsort($histogram);  
        return $histogram;  
    }  
}
```

```
0 references | 0 overrides  
10 public function __construct($startingUserNum) {  
11     $this->startingUserNum = $startingUserNum;  
12 }  
13  
2 references | 0 overrides  
14 public function collatzCalculate() {  
15     $startingUserNum = $this->startingUserNum;  
16     $listCollatz = [$startingUserNum];  
17  
18     while ($startingUserNum != self::REACHED_COLLATZ){  
19         if ($startingUserNum % self::EVEN_CASE_DIVIDE == 0) {  
20             $startingUserNum = $startingUserNum / self::EVEN_CASE_DIVIDE;  
21         }  
22         else {  
23             $startingUserNum = self::ODD_CASE_MULTIPLY * $startingUserNum + self::ODD_CASE_INCREASE;  
24         }  
25         $listCollatz[] = $startingUserNum;  
26     }  
27     return $listCollatz;  
28 }  
29  
30  
1 reference | 0 overrides  
31 public function intervalCalculate($startingUserIntervalNum,$sendingUserIntervalNum) {  
32  
33     $intervalCollatz = [];  
34  
35     $i = $startingUserIntervalNum;  
36     while ($i <= $sendingUserIntervalNum) {  
37         $this->startingUserNum = $i;  
38         $intervalCollatz[$i] = $this->collatzCalculate();  
39         $i++;  
40     }  
41     $this->cumulativeIntervalValuesStatistics = $intervalCollatz;  
42     return $intervalCollatz;  
43 }  
44  
45  
1 reference | 0 overrides  
46 public function gloriousStatistics($intervalCollatz) {  
47  
48     $iterationMinCount = null;  
49     $numberOfMinIterations = null;  
50  
51     $iterationMaxCount = null;  
52     $numberOfMaxIteration = null;  
53  
54  
55     $currentSelectedKeyHighestValue = 0;  
56     $numberOfKeyWithHighestValue = null;  
57  
58  
59 }
```

```
7 references | 1 implementation  
class userCollatz {  
    protected $startingUserNum,$startingUserIntervalNum,$sendingUserIntervalNum,$cumulativeIntervalValuesStatistics;  
}
```

Public modifiers were used so they can be used globally by the parent class for inheritance purposes. Private methods were not required, as all functionality needs to be accessible by both the parent and child classes. The protected modifier was used for class properties of userCollatz, allowing access within the parent class and its child class while still restricting direct external access.

Histogram testing

Using the interval range of 7 to 12, the following results were given:

```
Enter the first interval number for Collatz:7
Enter the second interval number for Collatz:12
Starting number: 7
Sequence step 1: 7
Sequence step 2: 22
Sequence step 3: 11
Sequence step 4: 34
Sequence step 5: 17
Sequence step 6: 52
Sequence step 7: 26
Sequence step 8: 13
Sequence step 9: 40
Sequence step 10: 20
Sequence step 11: 10
Sequence step 12: 5
Sequence step 13: 16
Sequence step 14: 8
Sequence step 15: 4
Sequence step 16: 2
Sequence step 17: 1

Starting number: 8
Sequence step 1: 8
Sequence step 2: 4
Sequence step 3: 2
Sequence step 4: 1

Starting number: 9
Sequence step 1: 9
Sequence step 2: 28
Sequence step 3: 14
Sequence step 4: 7
Sequence step 5: 22
Sequence step 6: 11
Sequence step 7: 34
Sequence step 8: 17
Sequence step 9: 52
Sequence step 10: 26
Sequence step 11: 13
Sequence step 12: 40
Sequence step 13: 20
Sequence step 14: 10
Sequence step 15: 5
Sequence step 16: 16
Sequence step 17: 8
Sequence step 18: 4
Sequence step 19: 2
Sequence step 20: 1
```

Figure 3.1

```
Starting number: 10
Sequence step 1: 10
Sequence step 2: 5
Sequence step 3: 16
Sequence step 4: 8
Sequence step 5: 4
Sequence step 6: 2
Sequence step 7: 1

Starting number: 11
Sequence step 1: 11
Sequence step 2: 34
Sequence step 3: 17
Sequence step 4: 52
Sequence step 5: 26
Sequence step 6: 13
Sequence step 7: 40
Sequence step 8: 20
Sequence step 9: 10
Sequence step 10: 5
Sequence step 11: 16
Sequence step 12: 8
Sequence step 13: 4
Sequence step 14: 2
Sequence step 15: 1

Starting number: 12
Sequence step 1: 12
Sequence step 2: 6
Sequence step 3: 3
Sequence step 4: 10
Sequence step 5: 5
Sequence step 6: 16
Sequence step 7: 8
Sequence step 8: 4
Sequence step 9: 2
Sequence step 10: 1
```

Figure 3.2

```
The number with Maximum Iterations:
Number:: 9
The maximum count of iterations:: 19

The number with the Least Iterations:
Number:: 8
iterationMinCount: 3

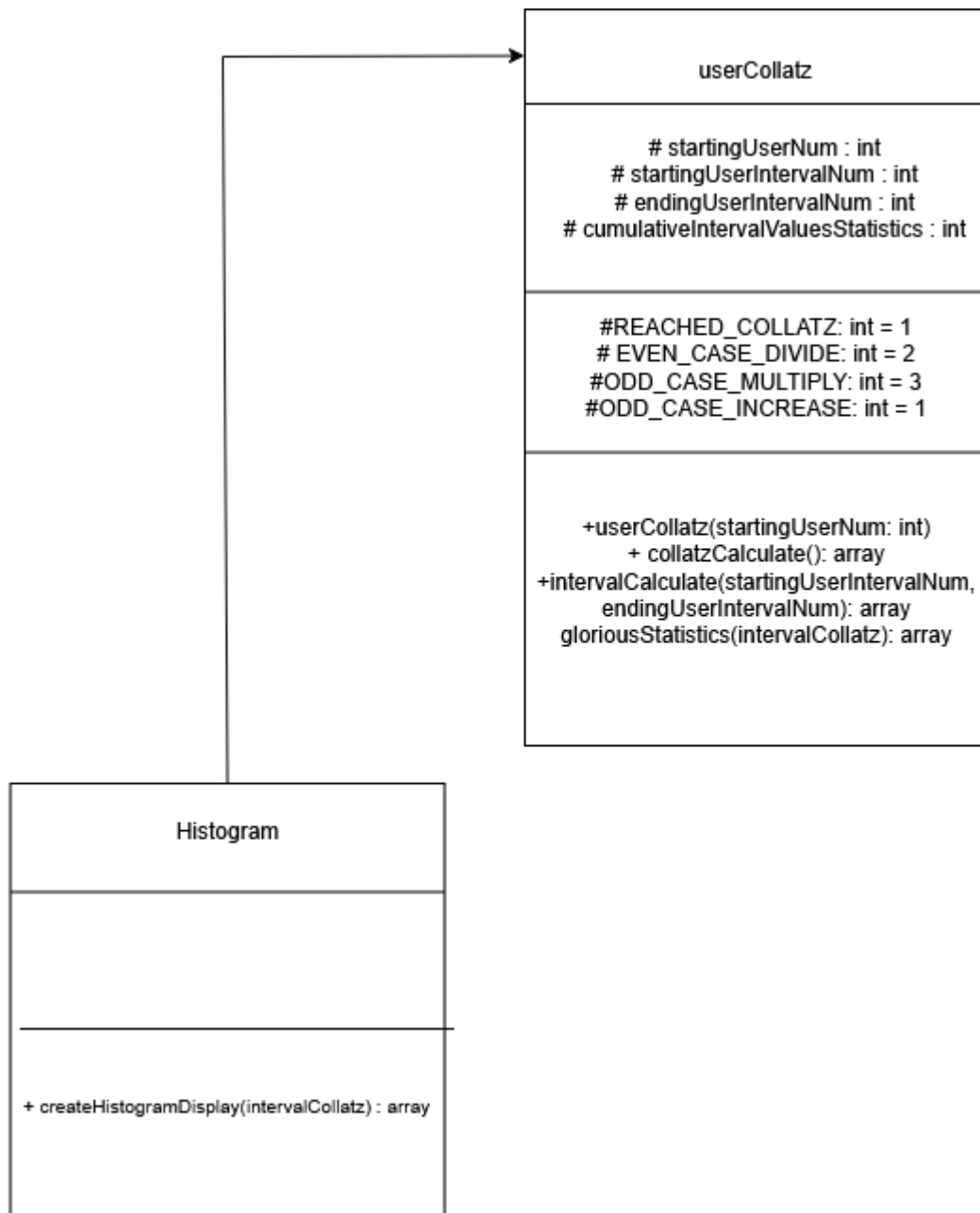
The number with the highest value:
Number:: 7
The highest value is:: 52

16 - 1
3 - 1
19 - 1
6 - 1
14 - 1
9 - 1
.....
```

Figure 3.3: Histogram results

Results: As shown in figure 3.3, the histogram is correctly calculated. Therefore, the Histogram class successfully inherits from the userCollatzClass and accesses the interval calculation results from its parent. It returns the frequency distribution of Collatz sequence lengths within the interval and displays them in descending order.

UML Diagram



References:

Microsoft. (n.d.). Visual Studio Code [Computer Software]. From: <https://code.visualstudio.com>